Python and Git 101

Python Programming and Data Analysis · Day 1 · Dataminded Academy

dataminded

NICE TO MEET YOU

Jan Vanbuel

Your trainer today. I like Rust, playing the saxophone and triathlon.

Platform engineer and Dataminded Academy lead



AI is a multiplier of experience

your experience × AI = what you get

Zero experience gives AI nothing to multiply.

Speak the language

Know the words a specialist uses, so you can ask for exactly that

Understand the tradeoffs

There are several ways to do it, each with a cost. You choose

Ask specific questions

A precise request gets code you can read and check

This prompt will make sense by 15:00

```
Write a function `summarize_run(path: Path) -> dict` that reads a CSV
log with `csv.DictReader`, converts `pressure_bar` to float, skips and
counts rows that raise `ValueError`, and returns the mean and max. Use
a list comprehension where it fits. Standard library only, no classes.
Add a pytest test using `pytest.approx`.
```




By 17:00 you can do six things on your own laptop



Run a script

Set up Python and run a script from PowerShell



Manage a project

A virtual environment per project, packages with uv



Write a program

Functions, conditions, loops and imports, run from VS Code



Read an error

Find the line a stack trace points to



Automate files

Read a folder of files and write a summary



Work with Git

Clone, commit on a branch, push and pull from Azure DevOps

Agenda

01	09:00 Setup check
02	09:30 Python on your laptop, scripts and notebooks
03	10:15 Language fundamentals, lunch at 12:30
04	13:15 Files, the standard library and exceptions
05	14:20 Projects with uv
06	15:05 Version control and Git
07	16:50 Wrap-up



Check the tools, then clone the repository

1 · The tools answer

```
uv --version
uv python list --only-installed
code --version
git --version
```

2 · The course material is on your laptop

```
cd $HOME\Documents
git clone https://github.com/datamindedacademy/ac-python-and-git-101
cd ac-python-and-git-101
uv sync
code .
```



Something fails?

Do not debug alone. Call me over: setup problems are normal on managed laptops.



Python on your laptop



Python fills the gap between shell scripts and C

Shell scripts

Quick to write. Painful once you need real logic, numbers or data structures

Python

Reads like a script, with lists, dicts, functions and a large library built in

C

Can do everything. Slow to write, and easy to get wrong



Photo: Daniel Stroud, CC BY-SA 4.0

Guido van Rossum started it in 1989 at CWI in Amsterdam, for system administration work that was too much for a shell script and not worth a C program. First release in 1991.

Python is the language, CPython runs it

Your .py file → **Compiled to bytecode** → **Run by the virtual machine**

> The **interpreter** is the program that does the last two steps: `python.exe`.

	CPython	PyPy	MicroPython
What it is	The reference implementation, written in C	Compiles hot loops to machine code while running	A small Python for microcontrollers
When you meet it	Always: it is what uv installs	Long pure-Python calculations	Sensors and embedded boards



Check which Python a command really runs

```
Get-Command python
```

```
CommandType  Name        Source
-----------  ----        ------
Application  python.exe  C:\Users\you\AppData\Local\Microsoft\WindowsApps\python.exe
```

Watch out

A `WindowsApps` path is the Microsoft Store shortcut, not a real Python. It opens the Store or does nothing useful.

Go to the repository, then uv run the script

```
# exercises/python/01-first-script/hello.py
print("Hello, test engineer!")
```

```
cd $HOME\Documents\ac-python-and-git-101
uv run python exercises/python/01-first-script/hello.py
```

```
Hello, test engineer!
```




VS Code runs the same script in three steps

1

OPEN THE FOLDER

File, Open Folder, choose `ac-python-and-git-101`

2

SELECT THE INTERPRETER

Command Palette, Python: Select Interpreter, pick the one in `.venv` or the uv Python 3.13

3

RUN

Play button at the top right, or the same `uv run python` command in the integrated terminal



Scripts and notebooks



A script is a text file that runs top to bottom

```
# bench_summary.py
pressure_bar = 7.5

power_kw = 37.5
print(f"Power: {power_kw} kW at {pressure_bar} bar")

print("Done")
```

```
Power: 37.5 kW at 7.5 bar
Done
```

Scripts repeat, notebooks explore

	Script (.py)	Notebook (.ipynb)
Runs	Start to finish, every time, on demand or scheduled	Cell by cell, in any order you click
Great for	Automation, reusable tools, anything others run	Exploring data, plotting, explaining results
Weak at	Slow feedback when exploring	Repeatability and testing
In Git	Clean text, easy to compare versions	JSON with embedded output, noisy differences



A notebook can run code you no longer see

```
In [1]   limit_bar = 9.0
```

```
In [2]   limit_bar = 6.0
```

ran, then deleted

```
In [3]   print(limit_bar)
```

```
Out      6.0
```

not 9.0

- You ran cell 1, then cell 2, then edited cell 2 away and forgot about it
- Cell 3 prints a value that no longer appears anywhere in the visible code
- A colleague runs the notebook top to bottom and gets a different answer



Exercise 01: First script

`exercises/python/01-first-script`

Words you can now use script · notebook · cell · kernel · hidden state · reproducible · interpreter · CPython · terminal · PATH · working folder · uv run

Language fundamentals

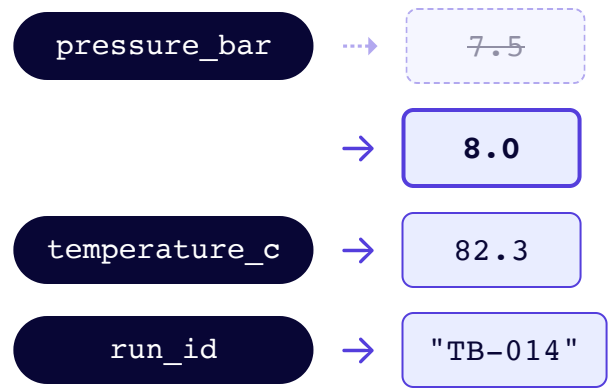


Variables are names for values

```
pressure_bar = 7.5
temperature_c = 82.3
run_id = "TB-014"

pressure_bar = 8.0
print(pressure_bar)
print(temperature_c)
```

```
8.0
82.3
```



Every value has a type

```
run_id = "TB-014"        # str: text
cycles = 12              # int: whole number
flow_l_s = 18.4          # float: decimal number
is_stable = True         # bool: True or False
operator = None          # None: no value (yet)

print(type(flow_l_s))
```

```
<class 'float'>
```

Text that looks like a number is still text

```
text = "7.5"
pressure_bar = float(text)
print(pressure_bar + 1)        # 8.5

count = 3
print("Cycles: " + count)
```

```
TypeError: can only concatenate str (not "int") to str
```




A traceback is a map to the error

```
Traceback (most recent call last):
  File "convert.py", line 2, in <module>
    pressure_bar = float(text)
ValueError: could not convert string to float: 'seven'
```

- Line 1 is only a header, it says: read on
- The middle shows the file, the line number and the code that failed
- The last line names the error type and what went wrong. Read it first.



f-strings put values inside text

```
run_id = "TB-014"
pressure_bar = 7.5
print(f"Run {run_id}: {pressure_bar:.1f} bar")
print(f"{pressure_bar * 14.5038:.0f} psi")
print("TB" in run_id, run_id.lower())
print("12.5 kW".split())
```

```
Run TB-014: 7.5 bar
109 psi
True tb-014
['12.5', 'kW']
```




Put the unit in the name

Written fast

```
p = 7.5
t = 82
calc(p, t)
```

Read easily

```
pressure_bar = 7.5
temperature_c = 82
to_kelvin(temperature_c)
```

Put the unit in the name. Constants in capitals: `ALARM_LIMIT_BAR = 8.5`



Exercise 02: Variables and types

`exercises/python/02-variables-and-types`

Words you can now use variable · assignment · type · string · float · f-string · convert · traceback



A list keeps values in order

```
pressures_bar = [6.8, 7.1, 7.5, 7.4, 7.9]
print(pressures_bar[0])
print(len(pressures_bar))
pressures_bar.append(8.2)
print(max(pressures_bar))
print(sorted(pressures_bar, reverse=True))
```

```
6.8
5
8.2
[8.2, 7.9, 7.5, 7.4, 7.1, 6.8]
```




Indexing starts at zero

index	0	1	2	3	4
value	6.8	7.1	7.5	7.4	7.9
negative	-5	-4	-3	-2	-1

```
p = [6.8, 7.1, 7.5, 7.4, 7.9]
print(p[1:3], p[:2], p[-2:])
```

```
[7.1, 7.5] [6.8, 7.1] [7.4, 7.9]
```




A dict finds values by name

```
reading = {"run": "TB-014", "pressure_bar": 7.5}
print(reading["pressure_bar"])
reading["flow_l_s"] = 18.4
print(reading.get("power_kw"))
print(reading)
```

```
7.5
None
{'run': 'TB-014', 'pressure_bar': 7.5, 'flow_l_s': 18.4}
```




A list for a series, a dict for a record

	list	dict
Find by	Position, in order	Name, the key
Fits	A series of the same kind of value	One record with different fields
Example	`pressures_bar[2]`	`reading["pressure_bar"]`

> Copilot will pick one for you unless you say which you want.



Exercise 03: Collections

`exercises/python/03-collections`

Words you can now use list · index · slice · dict · key · value



if runs the first branch that is true

```
pressure_bar = 9.4
if pressure_bar > 9.0:
    print("ALARM: over limit")
elif pressure_bar < 5.0:
    print("Warning: low pressure")
else:
    print("OK")
```

```
ALARM: over limit
```

`pressure_bar > 9.0`

true: run this branch, skip the rest

`pressure_bar < 5.0`

not checked

`else`

not checked

Checked top to bottom, the first true one wins



for runs the block once per element

```
pressures_bar = [6.8, 7.1, 9.4]
for p in pressures_bar:
    print(f"{p:.1f} bar")

for i in range(3):
    print(i)
```

```
6.8 bar
7.1 bar
9.4 bar
0
1
2
```




Most scripts initialise, loop, test, collect

```
pressures_bar = [6.8, 7.1, 9.4, 7.4, 9.1]
high = []
total = 0.0

for p in pressures_bar:
    total += p
    if p > 9.0:
        high.append(p)

print(high, round(total / len(pressures_bar), 2))
```

```
[9.4, 9.1] 7.96
```

1. **INITIALISE**
   An empty list and a zero total
2. **LOOP**
   Take each value in turn
3. **TEST**
   Decide with an `if`
4. **COLLECT**
   Report at the end



A list comprehension is that loop on one line

The loop

```
high = []
for p in pressures_bar:
    if p > 9.0:
        high.append(p)
```

The comprehension

```
high = [p for p in pressures_bar if p > 9.0]
```

```
# transform every value
psi = [p * 14.5038 for p in pressures_bar]

# the same idea without the brackets
total_high = sum(p for p in pressures_bar if p > 9.0)
```

> One `for`, one `if`. Anything more, write the loop.



Exercise 04: Control flow

`exercises/python/04-control-flow`

Words you can now use condition · branch · loop · iterate · block · indentation · comprehension



A function names a piece of logic

```
def to_psi(pressure_bar):
    return round(pressure_bar * 14.5038, 1)

print(to_psi(7.5))
print(to_psi(10))
```

```
108.8
145.0
```



`return` passes a value on, `print` only shows it

```
def double_and_print(x):
    print(x * 2)

result = double_and_print(4)
print(result)
```

```
8
None
```

A function without `return` gives back `None`. Use `print` to show, `return` to pass on.

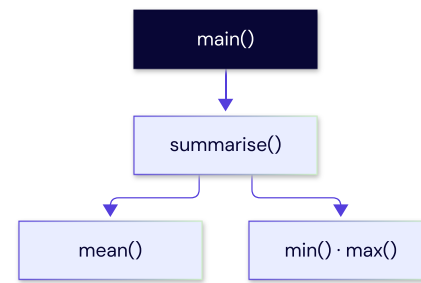
A script is small functions plus a main

```python
def mean(values):
    """Return the arithmetic mean of a list of numbers."""
    return sum(values) / len(values)

def summarise(values):
    return {"min": min(values), "max": max(values), "mean": mean(values)}

def main():
    pressures_bar = [6.8, 7.1, 9.4, 7.4]
    print(summarise(pressures_bar))

if __name__ == "__main__":
    main()
```



Type hints tell you how to call a function

```
def to_psi(pressure_bar: float, decimals: int = 1) -> float:
    return round(pressure_bar * 14.5038, decimals)
```

pressure_bar: float

The parameter and the type it expects

decimals: int = 1

A type and a default value, so you may leave it out

-> float

The type of the value that comes back

Hints are documentation that the editor and tools can check. Python does not enforce them when the script runs.

import gives you ready-made tools like math

```
import math
from math import sqrt

diameter_m = 0.05
area_m2 = math.pi * (diameter_m / 2) ** 2
flow_m3_s = 18.4 / 1000
print(round(flow_m3_s / area_m2, 2))

print(math.isclose(0.1 + 0.2, 0.3), sqrt(16))
```

```
9.37
True 4.0
```




Move code outward only when you need to

	Use it when	The cost
Lines in the script	You do it once, in a few lines	Copy and paste when you need it again
A function	You repeat it, or it deserves a name	A little structure to read through
A module	Several scripts need the same functions	One more file, and an `import`

> Copilot will pick one for you unless you say which you want.



A test says what correct means

```python
import pytest
import conversions

def test_bar_to_psi():
    assert conversions.bar_to_psi(1) == pytest.approx(14.5038)
```

Red: run, read the failure → Green: make it pass → Test first: write the next test

> This loop is **test-driven development**.



Exercise 05: Functions

`exercises/python/05-functions`

Words you can now use function · argument · return value · type hint · test · assertion · test-driven development · import · module



Files and the standard library



pathlib builds paths and finds files

```python
from pathlib import Path

data_dir = Path("data")
log_file = data_dir / "run_014.csv"

print(log_file.name, log_file.suffix, log_file.stem)
print(log_file.exists())

for f in sorted(data_dir.glob("*.csv")):
    print(f)
```

```
run_014.csv .csv run_014
True
data\run_014.csv
data\run_015.csv
```




with open reads a file and closes it again

```
from pathlib import Path

with open("data/run_014.csv", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

text = Path("data/run_014.csv").read_text(encoding="utf-8")
```

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```




Mode `w` overwrites, mode `a` appends

```
lines = ["run,max_pressure_bar", "TB-014,7.9"]
with open("summary.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")

with open("summary.txt", "a", encoding="utf-8") as f:
    f.write("TB-015,8.2\n")
```



Mode `"w"` replaces an existing file without asking. `"a"` adds at the end, `"r"` only reads.

csv turns each row into a dict of strings

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

```
import csv
with open("data/run_014.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[0])
print(float(rows[1]["pressure_bar"]) + 1)
```

```
{'t_s': '0', 'pressure_bar': '6.8', 'temperature_c': '71.2'}
8.1
```




json stores settings and nested data

```
import json
config = {"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": None}
text = json.dumps(config)
print(text)
back = json.loads(text)
print(back["channels"][0], back["operator"])
```

```
{"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": null}
pressure None
```

Exercise 06: Standard library

`exercises/python/06-standard-library`

Words you can now use standard library · path · glob · file handle · encoding · CSV · JSON



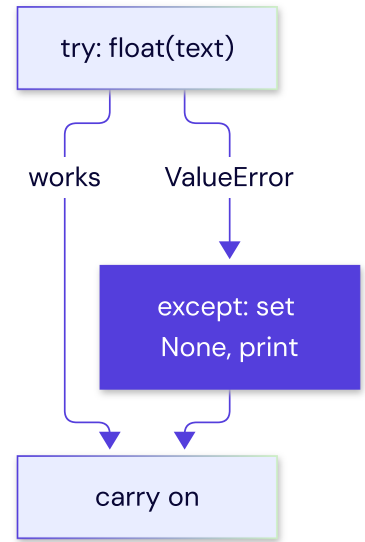
Exceptions



try / except handles an error you expect

```
text = "seven"
try:
    pressure_bar = float(text)
except ValueError:
    pressure_bar = None
    print(f"Skipping bad value: {text!r}")
print(pressure_bar)
```

```
Skipping bad value: 'seven'
None
```



Exercise 07: Files and exceptions

`exercises/python/07-files-and-exceptions`

Words you can now use exception · raise · catch



Projects with uv



uv and pip: the same job, fewer steps

	pip workflow	uv workflow
Environment	`python -m venv .venv`	`uv init`, then automatic
Activate	`.venv\Scripts\Activate.ps1`	Not needed: `uv run`
Install	`pip install matplotlib`	`uv add matplotlib`
Declare	`requirements.txt`, by hand	`pyproject.toml`, updated by uv
Pin versions	`pip freeze`, if you remember	`uv.lock`, automatic



Two commands give a project its own packages

```
uv init bench-tools
cd bench-tools
uv add matplotlib
uv run python main.py
```

```
Creating virtual environment at: .venv
Resolved 12 packages in 330ms
Installed 11 packages in 68ms
 + matplotlib==3.11.2
 + numpy==2.5.4
 ...
```




pyproject.toml declares, uv.lock pins

```toml
[project]
name = "bench-tools"
requires-python = ">=3.13"
dependencies = [
    "matplotlib>=3.11.2",
]
```

Commit

`pyproject.toml`, `uv.lock`, `.python-version` and your `.py` files

Do not commit

`.venv/`, big and machine specific



Exercise 08: A uv project

`exercises/python/08-uv-project`

Words you can now use virtual environment · package · dependency · lockfile · pin · sync



Python goes further, but not today

- **Styles**: procedural (today), object-oriented with classes and dataclasses, functional
- **Language**: decorators, generators, context managers
- **Debugging**: the VS Code debugger, with breakpoints and stepping, instead of `print`
- **Scripts that grow**: `logging` instead of `print`, arguments with `argparse`

- **Performance**: measure first (`timeit`, a profiler), then NumPy
- **Concurrency**: threads, processes, `asyncio`
- **Extending**: C, or Rust through PyO3
- **Packaging**: your own code, installable by colleagues



Version control



This prompt will make sense by 17:00

```
Create a branch `feature/max-temperature` from an up-to-date `main`. Stage
only `analyse_run.py`, commit with an imperative message, push and set the
upstream. Draft a pull request description from `git diff origin/main`. Do
not commit `output/`. If the merge conflicts, stop and show me both sides.
```




final_v3 is version control by hand

```
\\fileserver\compressors\testbench\
    analyse_run.py
    analyse_run_final.py
    analyse_run_final_v3_USE_THIS_ONE.py
```

With AI: `git diff` shows exactly what an assistant changed, so you keep what is good and undo the rest.

History

Which version made last week's report?

Who and why

Who changed the conversion factor?

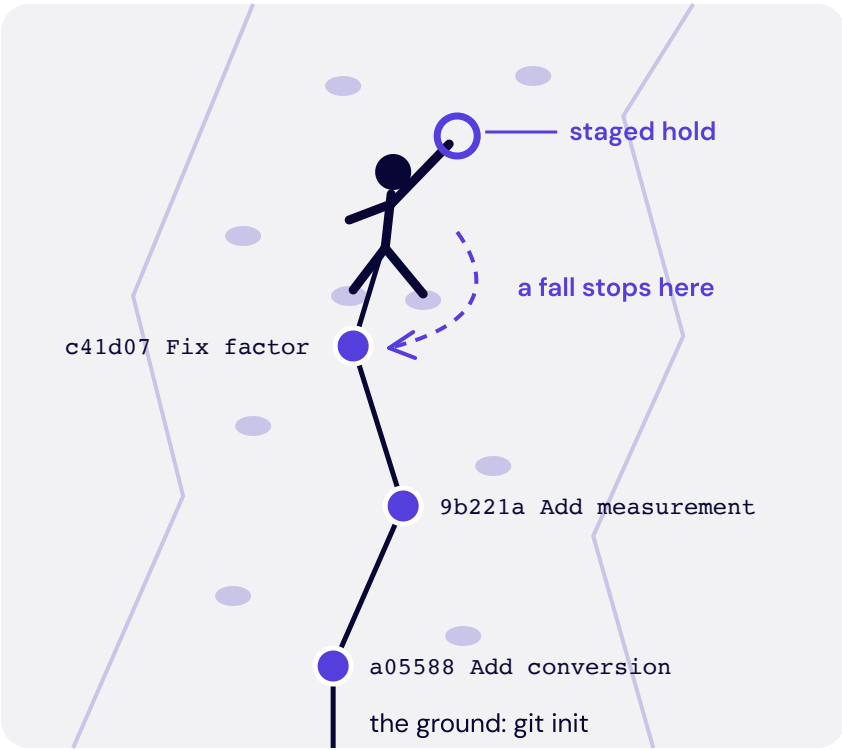
Undo

How do I get back to what worked?

Collaborate

Two people edit, whose version wins?

Stage a hold, then hammer in an anchor



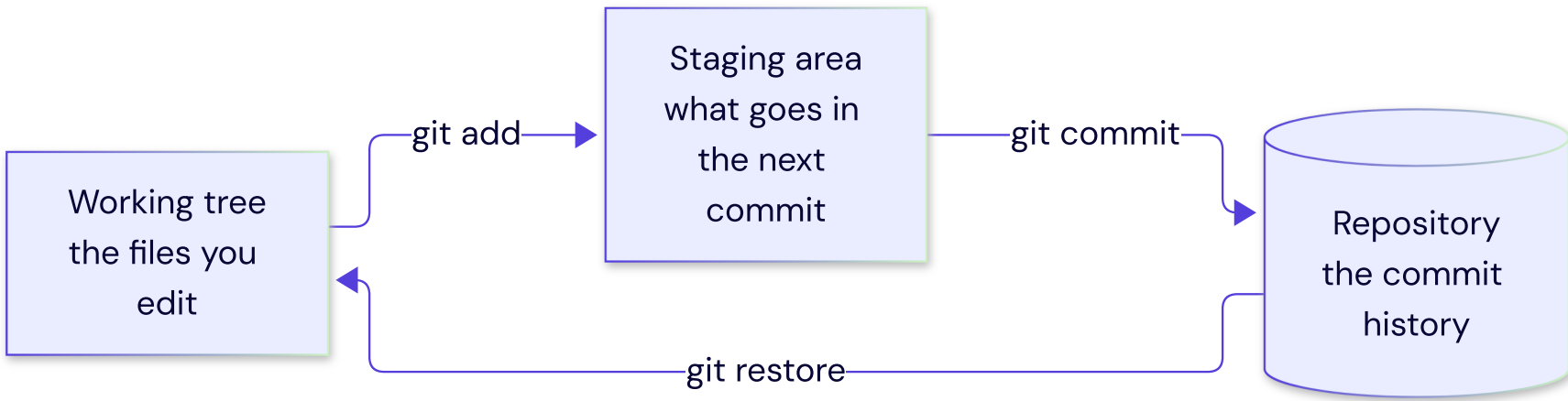
Working tree
You try moves above the last anchor. Nothing is saved yet.

Stage `git add`
You pick the holds for the next move.

Commit `git commit`
You hammer in an anchor and clip the rope. A fall takes you back to it, not to the ground.

History
The line of anchors below you.

A change moves through three areas



- You choose what goes into a commit by **staging** it first (the staging area is also called the **index**)
- `git status` tells you which area each change is in

A hash is the name Git gives to content

a05588…
files: snapshot `7c1e…`
parent: none
Jan · 3 March · "Add conversion"

←

9b221a…
files: snapshot `e02b…`
parent: `a05588…`
Sofie · 5 March · "Add measurement"

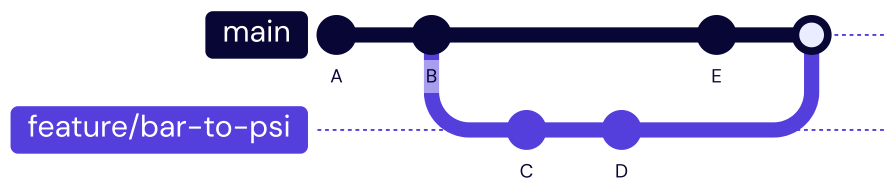
←

c41d07…
files: snapshot `41fa…`
parent: `9b221a…`
Jan · 6 March · "Fix factor"

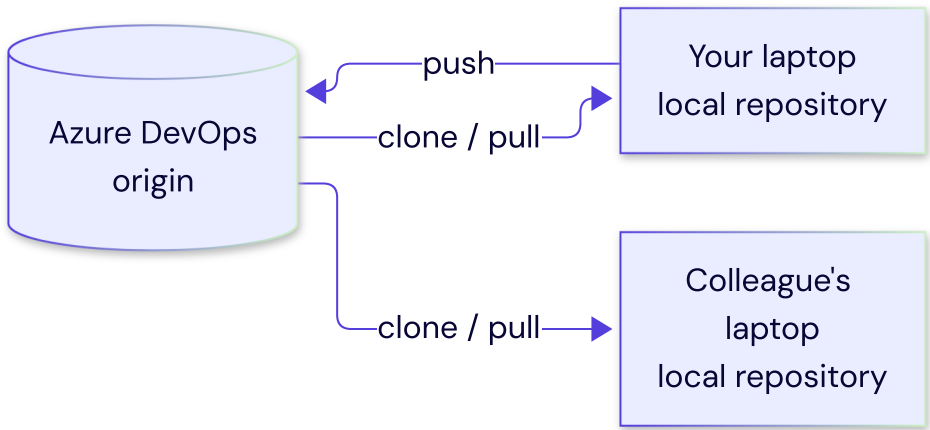
- Git stores every file and every commit under the **hash of its content**: 40 characters, calculated with SHA-1
- Same content, same hash. Change one character and the hash changes
- A commit contains its parent's hash, so changing an old commit changes every hash after it
- The first 7 characters are enough to point at a commit: `git show c41d07a`

A branch is a movable label on a commit

- **Repository**: a project folder plus its full history
- **Commit**: a saved snapshot with a message, author and date
- **Branch**: a movable label on a commit
- Work on a branch does not touch `main` until you merge it



Azure DevOps hosts the shared remote



- **Git** is the free tool on your laptop, **Azure DevOps** hosts the shared copy
- A **remote** is that shared copy, by convention called `origin`

Words you can now use repository · commit · branch · remote · working tree · staging area

Git on the command line



Set your identity once, then init a repository

```
git config --global user.name "Your Name"
git config --global user.email "you@atlascopco.com"
mkdir testbench
cd testbench
git init -b main
git status
```

- Identity: once per laptop, same email as your Azure DevOps account
- `git status` never changes anything: run it all the time

Stage what belongs together, then commit

```
git add conversions.py
git commit -m "Add psi to bar conversion"
```

```
[main (root-commit) a055887] Add psi to bar conversion
 1 file changed, 2 insertions(+)
```

- Message: short, imperative, one logical change: "Fix psi to bar factor", not "update"
- `git add .` stages everything: check `git status` first



log and diff show what changed

```
git log --oneline
git log --oneline --graph --all
```

```
9b221ad Add first measurement
a055887 Add psi to bar conversion
```

```
git diff            # edits not yet staged
git diff --staged   # what the next commit will contain
```

- `--graph --all` draws every branch as lines: you will read it in the branches exercise
- If the output is long, Git opens a pager: **q** leaves it



Exercise time: first commit

`exercises/git/01-first-commit`

About 20 minutes: `uv run python exercises/git/setup_exercise.py 1`



show and restore bring back old versions

```
git show a055887                    # one commit: message and diff
git show HEAD~1:conversions.py      # a file as it was one commit ago
git restore --source=HEAD~1 conversions.py
git restore conversions.py          # discard edits since last commit
```

- `HEAD` is the latest commit, `HEAD~1` the one before it
- `git restore <file>` throws your edits away: there is no undo for that



Extra exercise: history

`exercises/git/02-history`

When you are ahead, or after class: `uv run python exercises/git/setup_exercise.py 2`



Work on a branch, then merge it into main

```
git switch -c feature/bar-to-psi
git add conversions.py
git commit -m "Add bar to psi conversion"
git switch main
git merge feature/bar-to-psi
```

```
Updating 9b221ad..1cd9279
Fast-forward
```

- **Fast-forward** if `main` did not move; otherwise Git makes a merge commit
- `git branch` lists branches, `git branch -d <name>` deletes one



A conflict means Git needs you to decide

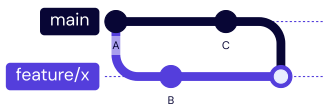
```
CONFLICT (content): Merge conflict in conversions.py

def psi_to_bar(psi):
<<<<<<< HEAD
    return psi * 0.068948
=======
    return psi * 0.06895
>>>>>>> feature/a
```

- A conflict is Git asking you to decide, nothing is lost
- Edit the file, delete the marker lines, then `git add` and `git commit`
- Changed your mind? `git merge --abort`

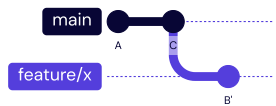
Rebase replays your commits on top of main

git merge main



Keeps both lines, adds a merge commit.

git rebase main



Replays your commits after the latest `main`: a straight line, new hashes.

```
git switch feature/x; git rebase main
git add <file>; git rebase --continue   # after fixing a conflict
git rebase --abort                      # give up
```

> Rebase only commits you have not pushed. Rebasing rewrites them, and colleagues who pulled the old ones get duplicates.

Exercise time: branches

`exercises/git/03-branches`

About 20 minutes: `uv run python exercises/git/setup_exercise.py 3`

Words you can now use stage · commit message · diff · HEAD · merge · fast-forward · conflict · rebase



Remotes and Azure DevOps



Clone copies the repository and its history

```
git clone https://dev.azure.com/<organisation>/<project>/_git/<training-repo>
cd <training-repo>
git remote -v
```

```
origin  https://dev.azure.com/<organisation>/<project>/_git/<training-repo> (fetch)
```

- The first time, a browser window asks you to sign in (Git Credential Manager)
- `clone` creates the remote `origin`; the URL is on the **Clone** button in Azure DevOps

Push uploads commits, pull downloads them

```
git push -u origin feature/bar-to-psi   # first push of a new branch
git push                                 # later pushes
git pull                                 # fetch and merge new commits
git fetch                                # only download, change nothing
```

- `-u` sets the upstream: it links your local branch to the remote one, needed only once
- `origin/main` is your laptop's last known copy of `main` on the server, updated by `fetch` and `pull`
- `pull` is `fetch` plus merge: commit or restore your edits first

Exercise time: Azure DevOps remote

`exercises/git/04-azure-devops-remote`

About 15 minutes: clone, branch, commit, push



A pull request lets a colleague review first

Push → Create → Review → Approve → Complete

Propose

A pull request (PR) proposes merging your branch into `main`. Colleagues read the diff first.

Complete

Pick the merge type (squash keeps `main` readable) and tick **Delete source branch**.

Update your laptop

`git switch main`, then `git pull`

Exercise time: pull request

`exercises/git/05-pull-request`

About 20 minutes: PR into main, review, complete, pull



.gitignore keeps files out of history

```
# .gitignore
.venv/
__pycache__/
data/raw/
```

- One pattern per line; commit the file itself so everyone shares it
- Never commit passwords, tokens or large raw data
- Day 2 uses it for data files and notebooks

Three tools drive the same Git

	Command line	VS Code	SourceTree
Advantages	Every feature, same everywhere, easy to copy into notes	Already open, diff and conflict views built in	Clear graph of branches, point and click
Disadvantages	Commands to learn, easy to mistype	Hides the commands, fewer rare features	Extra install, less common on managed laptops
Use it when	You need full power, or help from a colleague or a doc	Daily edit, commit, push work next to the code	You want to browse a complex history visually

Demo: Git in VS Code

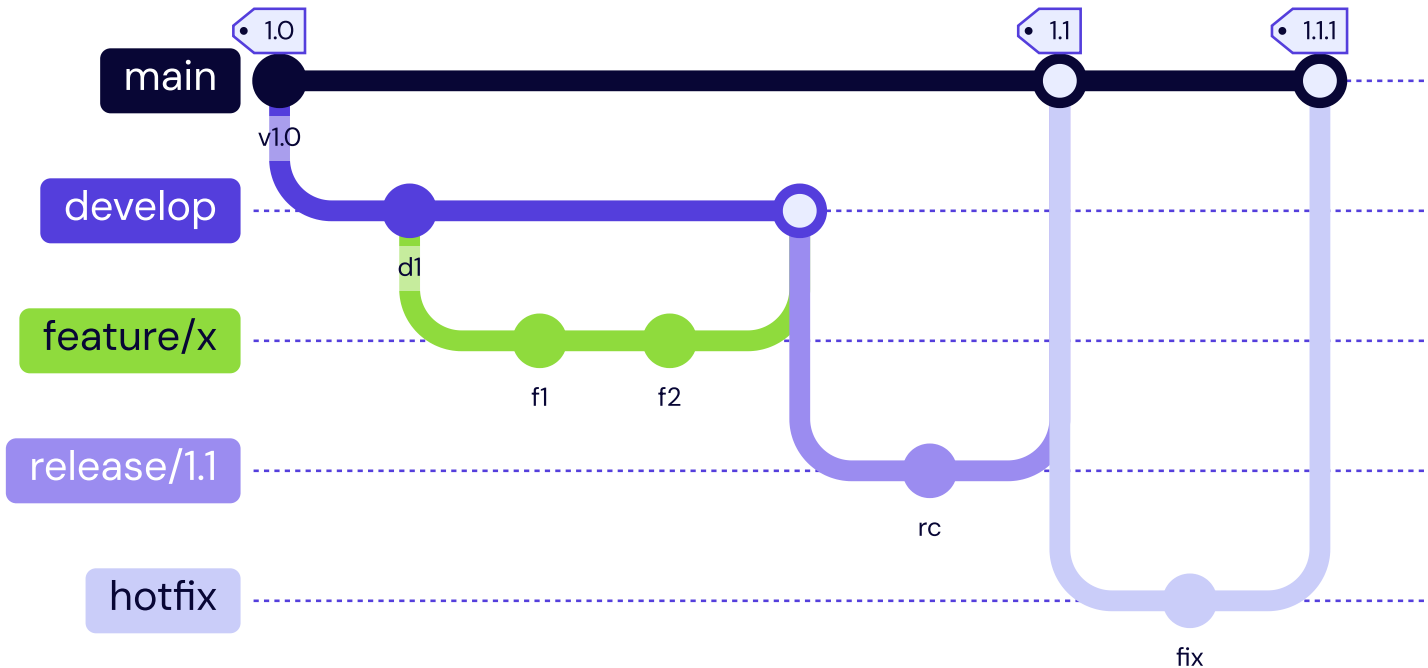
Optional practice: `exercises/git/06-git-in-vscode`

Words you can now use clone · origin · push · pull · fetch · upstream · origin/main · pull request · review

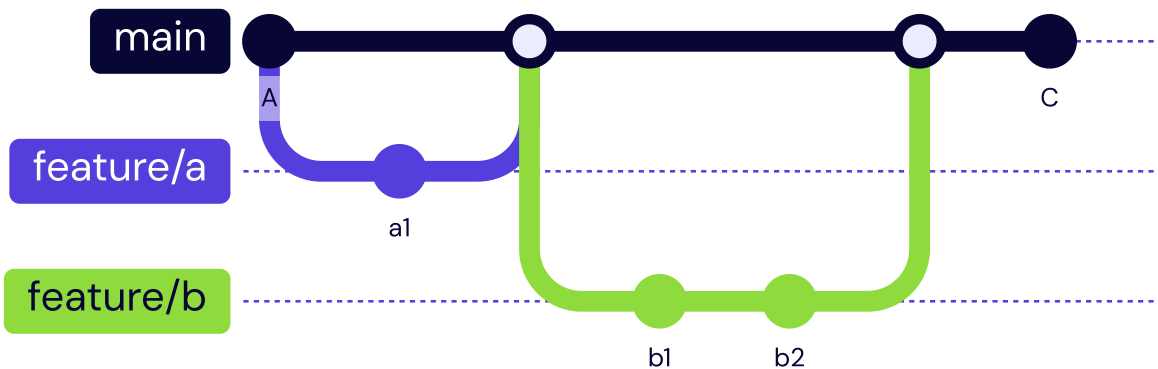
Branching models



GitFlow keeps a branch per release stage



Trunk-based: one main, short branches



- One long-lived branch, `main`; short branches merged through a pull request

Use trunk-based for scripts and analyses

	GitFlow	Trunk-based
Fits	Versioned products, several releases in support	Continuous delivery, scripts, analyses, small teams
Cost	Many branches, large merges, more conflicts	Discipline: small changes, review, tests

- **Use trunk-based**: short-lived `feature/...` branches and PRs; day 3 adds a branch policy on `main`
- Exists, not today: `git cherry-pick`, `git stash`

Words you can now use trunk-based · GitFlow · feature branch · short-lived · branch policy



You can now do all six things

Run a script

From PowerShell and VS Code

Manage a project

A .venv per project with uv

Write a program

Functions, loops, imports

Read an error

Find the line in a stack trace

Automate files

Read files, write a summary

Work with Git

Branch, commit, push, PR, pull

Today: the **vocabulary** and the tradeoffs. Day 3: you use them to direct and review Copilot and coding agents.

Read these two prompts again

Git

```
Create a branch `feature/max-temperature` from an up-to-date `main`. Stage only `analyse_run.py`, commit with an imperative message, push and set the upstream. Draft a pull request description from `git diff origin/main`. Do not commit `output/`. If the merge conflicts, stop and show me both sides.
```

Python

```
Write a function `summarize_run(path: Path) -> dict` that reads a CSV log with `csv.DictReader`, converts `pressure_bar` to float, skips and counts rows that raise `ValueError`, and returns the mean and max. Use a list comprehension where it fits. Standard library only, no classes. Add a pytest test using `pytest.approx`.
```




Eight commands cover daily Git

Local

```
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
git restore <file>
git switch -c <branch>
git merge <branch>
```

Remote

```
git clone <url>
git switch main
git pull
git switch -c feature/x
git push -u origin feature/x
# open the PR, complete it, repeat
```

Before day 2: one full cycle with a PR

Before day 2

- A working uv environment
- Redo the fundamentals you found hard
- One full clone, commit, push, pull cycle with a PR

Read more

- Pro Git book (`git-scm.com/book`)
- Learn Git Branching (`learngitbranching.js.org`)
- Python tutorial (`docs.python.org/3/tutorial`)
- uv docs (`docs.astral.sh/uv`)

Next days

- Day 2: pandas, NumPy, SciPy, Matplotlib, scikit-learn
- Day 3, going deeper: tests, linting, pipelines, coding agents and reviewing their work



More from Dataminded Academy



Containerization with Docker

Pack a script with its Python and packages, so it runs the same anywhere

`github.com/datamindedacademy/containerization_with_docker`



Orchestration with Airflow

Run scripts on a schedule, in order, with retries and alerts

`github.com/datamindedacademy/workflows_with_airflow`



Polars and PySpark

Data frames for large logs: Polars on one machine, PySpark on a cluster

`github.com/datamindedacademy/dataframe-of-mind`





Thank you

See you on day 2

